TrustNav: Explainable Per-Second Trust Scoring and Response for GNSS Spoofing and Jamming in Vehicle and Drone Fleets

Karan Deshmukh, Niranjan Satheesh Thandassery, Sudheer Reddy Jogi, Venkata Pavan Kumar Sanam
MSc Cybersecurity and Data Science, ESAIP École d'Ingénieurs, Angers, France
Group 2, technical project supervised by Sofiane Hamrioui
Corresponding author: domail2karandeshmukh@gmail.com

***Abstract*—Fleet software believes the satellite position its receivers report. Jamming and spoofing are now daily in parts of Europe, yet existing countermeasures either live inside one receiver and emit an unexplained flag, or monitor a fixed site. We present TrustNav, a software layer that scores every second, for each vehicle and drone in a fleet, how far its reported position can be believed (0–100), explains it in plain words, and *responds*: a position below the trust line is refused and replaced downstream by an estimate dead-reckoned from the odometer along the planned route. A noisy-OR fuses sixteen physical and signal rules, an unsupervised anomaly model trained on normal driving only, and Galileo Open Service Navigation Message Authentication (OSNMA). On ten held-out simulated scenarios over real Angers streets (3 800 observations, seven attack types), the combined detector catches every attack episode with a median delay of 1 s, at 99.3 % precision, 96.1 % recall and a 0.44 % false-alarm rate, including a relay-style drift that passes OSNMA; the response layer cuts the worst operator-visible position error from 8.0 km to 29 m. On a public recording of a real drive the rules raise no false alarm and the model must be re-calibrated on the receiver. On public recordings of real attacks (Jammertest 2024) the first version missed a phone captured by a spoofer for 22 min; a sixteenth rule, receiver time against the host clock, refuses every captured fix from the first second, and real jamming is alerted in 15 of 20 exposures. The system is open source.**

***Index Terms*—anomaly detection, dead reckoning, explainable AI, fleet telematics, Galileo OSNMA, GNSS jamming, GNSS spoofing, Isolation Forest, trust scoring.**

I. Introduction

Delivery vans, trucks, emergency vehicles, ships and drones rely on a Global Navigation Satellite System (GNSS) position and trust it without question. That trust is easy to break. GNSS signals reach the ground from about 20 000 km away and are extremely weak, so a small transmitter nearby can either drown them (*jamming*) or imitate them and feed the receiver a false but believable position (*spoofing*). Jammers, whose possession and sale are illegal in France, are regularly found in vehicles whose drivers want to escape fleet tracking [1]; in April 2017 one left running in a car parked at Nantes airport delayed several flights and cost its owner a €2 000 fine [2], and in July 2023 a truck driver's jammer disrupted signals at Lyon airport until the ANFR traced it [3]. Since 2022, interference around the Baltic Sea and near conflict zones has become a daily event for aviation, with about 1 500 flights spoofed per day by the summer of 2024 [4], [5].

Two gaps motivate this work. The fleet telematics platforms that operators use display the reported position and, at most, relay the receiver's own jamming flag [6]; they do not say whether the position is true. The detection products that exist either run inside one receiver and output a state code with no explanation [7], [8], or are fixed multi-antenna probes that protect a site rather than a moving platform [9]. The natural place for a defence is, we argue, a software layer beside the receiver, working on what any receiver already reports, that (i) gives each platform a per-second trust score, (ii) explains it in verifiable terms and (iii) acts on it so that nothing downstream consumes a lie. TrustNav is this layer. Our contributions are:

1) a hybrid detector that fuses sixteen explainable physical and signal rules, an unsupervised anomaly model trained on normal operation only, and Galileo OSNMA status, with a calibrated mapping from anomaly probability to risk;
2) a *response layer* that refuses untrusted fixes and substitutes a route-constrained dead-reckoned estimate from the odometer, with a rejoin rule that a spoofer cannot defeat by briefly looking clean;
3) an open-source implementation: engine, cloud back end, a console that explains every decision in plain words, and adapters for real receivers and for a phone;
4) an evaluation on disjoint routes over real Angers streets (event-level and per-second metrics, confidence intervals, baselines, sensitivity, what the response layer prevents) and on real recordings: a clean drive, and real jamming and spoofing, where the first version failed and a new rule came out of it.

II. Background and Related Work

A. Threats

A *jammer* transmits noise in the GNSS bands; the receiver's carrier-to-noise density ratio (C/N_0) collapses, satellites are lost and eventually no fix is computed. A *spoofer* transmits counterfeit satellite signals from a simulator; an asynchronous spoofer captures the receiver and moves the fix abruptly

("teleport"), a synchronous one aligns with the live signals first and pulls the fix away slowly ("lift-off" or "drift"). A *meaconing* or relay attack re-broadcasts genuine signals with a delay, so the navigation data remain authentic while the position is wrong. *Forged navigation data* fabricate the messages themselves. Each leaves different traces: impossible motion, signals stronger and more uniform than real satellites, disagreeing residuals, clock jumps, or failed authentication.

B. Galileo OSNMA

Galileo's Open Service Navigation Message Authentication signs the I/NAV navigation data with a TESLA delayed-disclosure scheme; its Initial Service was declared operational on 24 July 2025 [10], and EU regulations make it mandatory in the smart tachographs of heavy vehicles registered from 24 December 2025 [11]. It lets a receiver flag unauthenticated data, but the EU GNSS Service Centre states that it "does not prevent the occurrence of all kinds of spoofing, and does not protect against jamming" [12]; GPS is not covered, and replay and time-synchronisation attacks that pass OSNMA have been demonstrated on real receivers [13]: one kind of evidence, not a complete defence.

C. Existing countermeasures

Mass-market u-blox receivers report a spoofing state and a jamming state; the current ZED-F9P firmware adds a jamming detector with logging, a spectrum analyser and OSNMA [7]. The flags are deliberately cautious: at Jammertest 2025 both tripped on a current u-blox receiver during jamming-only runs [14]. Septentrio's AIM+ adds interference mitigation, spoofing detection and OSNMA inside its receivers [8]. GPSPatron's fixed probes stream raw observables to a cloud that classifies jamming and spoofing [9]. infiniDome's GPSdome nulls jammers with up to four antennas [15]. Geotab's newer GO devices log a jamming event from the GPS module [6]; no telematics vendor we reviewed claims spoofing detection. Table I summarises; TrustNav consumes receiver flags and OSNMA as inputs rather than competing with them.

D. Detection with learned models and consistency checks

Recent work applies learned models to receiver observables, mostly on the TEXBAT recordings [16]: an LSTM network with a conditional VAE reaches an F_1 of 96.9 % on TEXBAT only [17]; a convolutional VAE trained on clean data reaches a true-positive rate above 99 % at 2 % false positives [18]; a chi-square test on raw intermediate-frequency samples, which TrustNav cannot access, reports over 99 % accuracy with no false alarms [19]. FusionRipper showed that a patient drift defeats multi-sensor fusion on real traces [20]; hence our stealth-drift scenario. Closest to our setting, Kriezis *et al.* evaluated single-receiver checks on €200 u-blox ZED-F9P receivers against real spoofing in the eastern Mediterranean: C/N_0, residuals and clock drift were informative, gain control and Doppler not, and the SBAS MOPS interference algorithm scored 94 % on one receiver but below 13 % on another [21]. TrustNav belongs to the "train on normal data only" family of [18], but it is a *system*: it adds the route and the odometer as evidence a receiver cannot supply, reports per-second trust rather than a binary verdict, explains each decision, and responds.

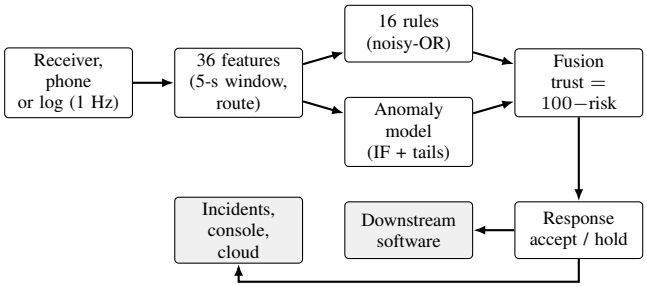


Fig. 1. Processing pipeline. Every observation (position, speed, signal statistics, clock, OSNMA status, odometer speed) becomes a feature vector, is judged by rules and by an anomaly model, fused into a trust score with evidence, and passed to the response layer, which decides which position downstream software receives.

III. System Design

A. Threat model and assumptions

The attacker transmits or relays radio signals near one platform: asynchronously (a simulator with a power advantage), synchronously (aligned with the live signals, lifting the fix off slowly), or by relaying authentic signals so that OSNMA still verifies. Its goal is a false position, abrupt or covert, the fleet acts on. We trust the odometer or CAN speed, the planned route and the authenticated channel to the cloud. Out of scope are multi-antenna and intermediate-frequency evidence, odometer tampering, and time-synchronisation attacks on OSNMA [13]. Table II maps the seven simulated attacks onto this model, names the evidence that reveals each, and gives the detection delay and the position errors measured in Section V.

B. Observations and features

Once per second the receiver reports position, speed, heading, satellites tracked, C/N_0 statistics, position dilution of precision (PDOP), pseudorange residuals, clock bias and drift, a jamming indicator, OSNMA status and, where available, an inertial or odometer speed (Fig. 1). A streaming extractor keeps a window of five observations and computes 36 features: distance moved, derived versus reported speed, acceleration and heading rate, distance to the route, C/N_0 mean, change and spread, satellite-count change, PDOP, residuals, clock deltas, the GNSS-versus-inertial speed mismatch, and explicit `missing_*` flags so that an absent input is never read as zero.

C. Rules

Sixteen deterministic checks compare a feature with a named threshold from the platform's physical profile (road vehicle: 60 m/s, 8 m/s^2, 90°/s, 250 m per second) or from signal physics (a 6 dB-Hz C/N_0 shift, PDOP above 5, residuals above 12 m, a clock-drift change above 60 ns/s, a jamming

TABLE I
HOW TRUSTNAV DIFFERS FROM WHAT EXISTS (SOURCES IN SECTION II; VENDOR CAPABILITIES AS THE VENDORS CLAIM THEM)

	Receiver flags (u-blox, Septentrio)	Fixed probes (GPSPatron)	Anti-jam hardware (infiniDome)	Fleet telematics (Geotab etc.)	OSNMA	**TrustNav**
Detects	jamming, spoofing	jamming, spoofing	jamming (mitigates)	relays the receiver's jamming flag	forged Galileo data	**jamming, spoofing, forgery**
Per-second score	no (state code)	probability	no	no	no	**yes, 0–100**
Explains why	no	partly (charts)	no	no	n/a	**yes, in words with values**
Responds	no	alerts	yes (RF nulling)	report	flag	**refuses the fix, estimates, recommends an action**
Extra hardware	none	probe, 1–3 antennas	antenna array	none	OSNMA-capable receiver	**none**

TABLE II
THE SIMULATED ATTACKS: CLASS, WHAT REVEALS THEM, THE DELAY FROM ATTACK START TO THE FIRST ALERT (RULES ALONE / COMBINED; "–": THE RULES NEVER FIRE) AND THE WORST POSITION ERROR DURING THE ATTACK, RAW FIX / PROTECTED ESTIMATE (SECTION V)

Attack	Class	Evidence	OSNMA	Alert	Error m
Teleport, 8 km jump	asynchronous	position jump (critical), route	not available	0 s / 0 s	8 003 / 3.8
Gradual drift <1 m/s	synchronous lift-off	C/N_0 shift, route, model	verifies	– / 1 s	211 / 16.3
Signal anomaly	spoofer onset	C/N_0 jump, uniformity, model	verifies	– / 1 s	6.8 / 20.8
Stealth drift	relay (meaconing)	model: route, speed mismatch	verifies	– / 42 s	179 / 20.1
Forged Galileo data	forgery	OSNMA failure (critical)	fails	15 s / 1 s	130 / 15.4
Jamming, 100 s	broadband noise	C/N_0 collapse, satellites lost, no fix, jam indicator	lost	12 s / 1 s	blank / 1.8
Drone off survey	synchronous lift-off	route, speed mismatch, model	verifies	– / 1 s	257 / 28.8

indicator above 80/255, satellite changes of five or more), plus OSNMA failed or lost, fix lost, and receiver time: its UTC may not leave the host clock by more than 5 s, nor run backwards against its own count (Section V-F). A rule's contribution c_i is its weight times a severity that rises from 0.5 at the threshold to 1.0 at three times the threshold; rules combine by noisy-OR,

$$r_{\text{rules}} = 100\left(1 - \prod\nolimits_i (1 - c_i)\right), \tag{1}$$

so that independent evidence reinforces itself. Three conditions are *critical*: a position jump above four times the profile limit, an OSNMA failure and a time jump; each pins the combined risk at 70 or more. Thresholds sit where behaviour becomes physically implausible: the rules stay almost silent on normal data at the cost of slow attacks.

D. Anomaly model

Because any supervised model would learn the shapes of the simulator's attacks, the learned component is trained on *normal operation only*. Two detectors complement each other: an Isolation Forest [22] (300 trees, scikit-learn [23]) finds unusual combinations of the 19 model inputs, and a per-feature tail detector finds a single improbable input, such as a fix 90 m off the road with clean signals, and names it. Each detector's raw score is converted to a tail probability p on a calibration set of normal runs not used for fitting (the fraction of normal observations at least as anomalous), the two are combined with a Bonferroni correction, $p = \min(1,\, 2\min(p_{\text{forest}}, p_{\text{tail}}))$, and the risk is

$$r_{\text{model}} = 100\,\min\left(1,\ \frac{-\log_{10} p}{4}\right), \tag{2}$$

so a risk of 50 corresponds to roughly a one-in-a-hundred event under normal operation. A trailing median over three seconds suppresses isolated spikes, and from a risk of 25 the model lists its most unusual inputs with their normal ranges.

E. Fusion, trust and incidents

Rules and model are two witnesses, not two votes to average (a model alarm of 90 averaged with quiet rules at 20 would give 55). The combined risk is a weighted noisy-OR,

$$r = 100\left(1 - (1 - w_r\, p_r)(1 - w_m\, p_m)\right), \qquad \text{trust} = 100 - r, \tag{3}$$

with $p_r = r_{\text{rules}}/100$ and $p_m = r_{\text{model}}/100$. The weights $w_r = 0.6$ and $w_m = 0.8$ were chosen by grid search on validation routes only (Section V-C). Trust bands are Trusted (≥ 75), Degraded (50–74), Suspicious (25–49) and Untrusted (< 25). An incident opens after three consecutive seconds below 50 and closes after five clean seconds.

F. Response layer

Detection alone leaves the operator looking at a lie; the response layer turns the score into an action, as intrusion *prevention* follows intrusion *detection*. A fix is *accepted* when it is present, its trust is at least 50 and no critical rule fired. Otherwise it is *held*: the fix is refused, and the position handed downstream is dead-reckoned from the last accepted fix using the odometer speed, which a spoofer cannot touch. With a route the estimate advances along it (map-matched), dropping the lateral error of the last accepted fix; without one it continues on the last accepted heading. A hold is *released* only after five consecutive acceptable observations whose fix lies within 60 m plus 3 % of the distance dead-reckoned while held, so a spoofer cannot end it by looking clean while still far away; a sixty-second clean streak releases with a ten times looser tolerance, the discrepancy reported, because an odometer estimate drifts too; a fix still kilometres away stays refused. Each observation also carries an action: CONTINUE, VERIFY (confirm by other means before acting) or ISOLATE (trust below 25 or critical evidence: stop using the position).

IV. Implementation

TrustNav is open source[1] and deployed as a public demonstration.[2] The **engine** is Python: simulator, feature extractor, rules, model, fusion, incidents, response layer, evaluation and live streamer, with 42 automated tests. Two **adapters** feed the same detector: one parses NMEA and u-blox UBX messages from a serial port or a log; the other turns a phone into a receiver through a web page that sends the phone's own fixes once a second, with a software attack overlay for demonstrations on real streets. The **cloud** side is Supabase (PostgreSQL with row-level security and Realtime channels): the browser can only read, writes pass through an edge function that checks a hashed token, and no secret is in the repository. The **console** is a Next.js application (live fleet map, replay, narrated explainer) with 109 end-to-end checks on every build. Every decision is explained twice: as the evidence lines the engine produced ("Fix moved 7 998 m in 1 s (limit 250 m)"), and as a verdict in everyday words derived from the evidence codes alone ("the satellite data is forged; TrustNav is tracking the van by its odometer"), so that it works unchanged on a real receiver. *Data and code:* everything, this paper included, is in the repository; `trustnav_engine.evaluate` and `export` regenerate Tables II and III, `paper/figures/make_figures.py` regenerates the figures and writes every quoted statistic to `analysis.json`, and `realcheck` and `attackcheck` reproduce Sections V-E and V-F.

V. Evaluation

A. Data and protocol

No public dataset provides labelled per-second receiver observables for fleets with routes and odometers, and transmitting attacks is illegal, so a deterministic, seeded simulator produces the observations a receiver would report. Road routes are real Angers street geometry routed with OSRM on OpenStreetMap data; drone missions are lawnmower surveys. Normal behaviour includes stops, cornering and urban canyons, the usual causes of false alarms. The seven attack overlays of Table II are labelled per observation. Routes have one role each and are never shared: five road loops and two drone areas for *training* (normal runs only, 10 785 observations for fitting and 5 630 for calibration), two loops and one drone area for *validation*, on which the fusion weights and nothing else were chosen, and two loops and one drone area for the *held-out* catalogue that produces every reported number: ten scenarios, 3 800 one-second observations, 1 550 under attack, 2 250 clean. Rule thresholds come from physics, not data. The per-second unit is the observation; an alert is a combined risk of 50 or more; an attack episode counts as detected if any of its seconds is alerted.

[1] https://github.com/domail2karandeshmukh-cyber/trustnav

[2] https://trustnav.vercel.app

TABLE III
Detection on the held-out catalogue (3 800 observations)

	C/N_0+sats	Rules only	Model only	**Combined**
Precision	100.0 %	100.0 %	97.9 %	**99.3 %**
Recall	9.4 %	27.9 %	90.1 %	**96.1 %**
F_1	0.172	0.437	0.939	**0.977**
False-alarm rate	0.00 %	0.00 %	1.33 %	**0.44 %**
ROC AUC	–	0.857	0.937	**0.990**
Precision–recall AUC	–	0.832	0.951	**0.991**
True positives	146	433	1 397	1 489
False positives	0	0	30	10
False negatives	1 404	1 117	153	61
True negatives	2 250	2 250	2 220	2 240

B. Detection

Table III compares the three ways to decide with a MOPS-style baseline that alerts when the C/N_0 mean falls below 35 dB-Hz or fewer than five satellites are tracked [21]. Rules never raise a false alarm but miss slow attacks; the model catches subtle change but is noisier; combined, each covers the other's blind spot; the baseline sees only jamming. Because seconds within a run are correlated, we also bootstrap over scenarios (2 000 resamples): the 95 % intervals are 97.9–100 % for precision, 89.4–99.6 % for recall, 0.940–0.997 for F_1 and 0–1.0 % for the false-alarm rate. At the event level all seven attack episodes are detected, with a median delay of 1 s and a maximum of 42 s (Table II); the ten false-alarm seconds form four episodes of 1–5 s in 37.5 min of clean operation (16 s or 6.4 episodes per hour); three are shorter than the three-second incident rule and the fourth is the recovery tail of the jamming run, inside the incident the jamming had already opened, so no false incident is raised. The stealth drift, invisible to every rule and to OSNMA, is caught by the model after 42 s at roughly 12 m off the road, where the fixed 150 m route rule alone would need about 3.5 min; trust briefly climbs back whenever that drift pauses (Fig. 2b), but the hold does not release, because the fix is still far from the estimate.

C. Sensitivity, calibration and ablation

Fig. 3 shows that the operating point is not tuned to the test set. Sweeping the alert line from 25 to 75 moves F_1 between 0.86 and 0.98 with its maximum at the chosen 50, while the false-alarm rate falls from 4.0 % to 0.1 %; a line at 60 halves false alarms (0.22 %) for four points of recall. The combined risk is calibrated: 1.8 % of the seconds scored 0–25 are under attack, 20.6 % at 25–50, 97.5 % at 50–75 and 99.8 % above 75. On the validation routes $w_m = 0.8$ is best for every w_r, and $w_m = 1.0$ doubles the false-alarm rate for the same F_1. Noisy-OR assumes conditionally independent evidence, which jamming violates (C/N_0, satellites and the jamming indicator fall together): the risk is then over-confident, harmless for an alert but not as a probability.

D. What the response layer prevents

Because the simulator knows the true position, we measure during each attack the worst position an ordinary fleet screen would show (raw fix) against the worst the response layer

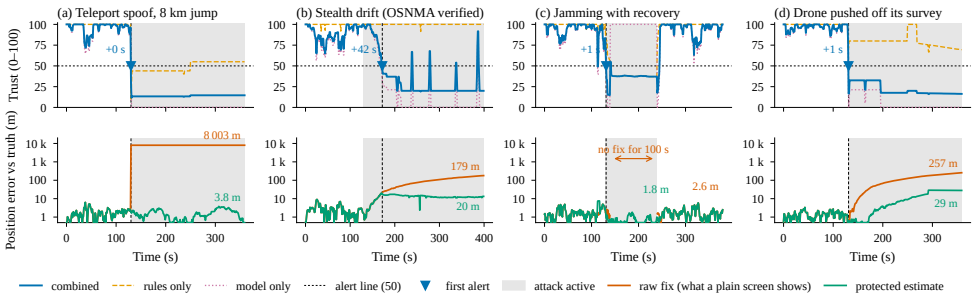


Fig. 2. Four held-out scenarios, second by second. Top: trust from the combined detector and from each witness alone; marker and dashed line: the first alert and its delay. Bottom: position error against ground truth for the raw fix an ordinary screen would show and for the estimate handed downstream (log scale, maxima labelled). In (b) no rule fires and OSNMA still verifies; the model alone catches the drift.

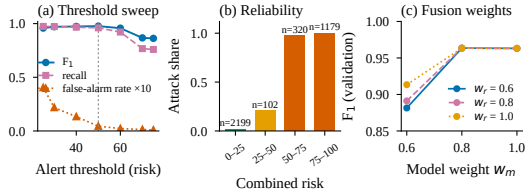


Fig. 3. (a) F_1, recall and false-alarm rate against the alert line (dashed: the chosen 50). (b) Share of seconds under attack per combined-risk band, with the seconds per band. (c) Validation F_1 over the fusion-weight grid.

handed downstream (Table II, last column; Fig. 2 bottom). Across the spoofing scenarios the worst reported error is 8 003 m and the worst protected error 29 m; during the 100 s jamming outage an ordinary screen is blank while the estimate stays within 2 m. These are worst cases of one scenario each and assume an honest odometer. The cost is small: a hold adds a few metres of odometer error even when the attack did not move the position (signal anomaly, 20.8 m), and holds outside any attack (a brief dip on a normal run, the seconds after an attack ends before the fix rejoins) total 26 s over the catalogue.

E. A real drive without attacks

A public 27.6-min recording of a u-blox ZED-F9K driving a loop in Mountain View, with a NovAtel SPAN reference trajectory [24], was replayed through the same adapter and detector; the reference track served as the route and its speed as the odometer. The rules alone raised no alert in the whole drive (minimum trust 90); the C/N_0+sats baseline raised 86 false-alarm seconds; the model trained on the simulator alerted on 89 % of the seconds. Re-calibrated on the first 19 min, it raised no alert in the remaining 8.5 min, and software attacks overlaid on that segment (an 8 km teleport, a 0.7 m/s drift, jamming) were caught after 0, 35 and 1 s, with worst protected errors of 0.8, 24 and 0.8 m (idealised: the route is the driven path).

F. Real attacks: Jammertest 2024

The VAARA dataset [25] was recorded in a car at Jammertest 2024 (Andøya, Norway), where the authorities let jammers and spoofers transmit: a u-blox receiver (NMEA) and a phone whose log stamps each sentence with the phone's own system clock, with the test times. Both were replayed unchanged.

Spoofing. In test 2.6.2 a spoofer captured the phone for 26.5 min: its time went back an hour and its position 34 m aside; when the car drove off the phone stayed where the spoofer had put it and ended 10.2 km from the truth. The u-blox beside it was not captured and is the reference. The fifteen rules first published alerted after 21 min 49 s and refused 5 of 909 captured fixes: nothing in the spoofed stream is physically impossible. An odometer or the re-calibrated model raised that to 32 and 41. What the spoofer could not hide was time. With the sixteenth rule the alert comes in the same second and all 909 fixes are refused, with no alert in the 37 min before the capture or the 9 min declared free of tests; the estimate handed on stays within 45 m (median 2.3 m; a best case, reference speed as odometer and track as route).

Jamming. The u-blox drove past parked multi-band jammers and started inside a jammed area: 20 exposures, 1 497 s. The rules alerted on one, for want of a jamming indicator in NMEA; the hybrid, its model re-calibrated on the 38 min recorded before the first test, alerted on 15, about 10 s in, and showed all 20 as degraded, with 0.8 % false alerts on clean-signal seconds. Beside the spoofer the uncaptured u-blox lost up to 30 dB of C/N_0: it was shown as degraded for 99 % of 62 min, and kept. With so little calibration data the model's risk cannot exceed 69: it alerts on exposures, not on every jammed second, and a lost fix alone is held, not alerted. The recordings also exposed four defects in the adapter and the response layer, now fixed.

VI. Discussion and Limitations

The simulator numbers are an upper bound. One simulator produces every split, so the model's "normal" is the simulator's; the attacks are textbook versions, and an adversary who matches C/N_0 spread and clock behaviour, or updates ephemeris within seconds of the live signal as real spoofers do [21], would be harder to detect. Route deviation assumes an assigned route (delivery fleets, survey drones), not a free driver. OSNMA is simulated, multi-antenna and intermediate-frequency evidence are absent, and the response layer is only

as good as its odometer. The receiver dependence reported by Kriezis *et al.* [21], and seen in Sections V-E and V-F, means the learned normal must be re-calibrated on the receiver used; the real attacks are two days of one campaign, and the time rule needs a host clock, or a spoofer that moves time backwards.

What remains: benign drives around Angers with the phone adapter and a receive-only u-blox receiver, for a false-alarm rate over hours rather than minutes and for re-calibration on the receiver actually used; raw I/Q captures (FGI Jammertest 2023 [26], TEXBAT [16]) through a software receiver; then OBD-II speed, a second cheap receiver against synchronous spoofing, real OSNMA from raw Galileo pages, and map matching without an assigned route.

VII. Safety, Legal and Ethical Considerations

No signal is transmitted at any point: every attack exists as software. Possessing or selling a jammer is illegal in France [1]; real attacks are replayed from recordings made by others under licence, and the Phase 2 hardware is receive-only. A real deployment tracks vehicles and therefore their drivers, so purpose limitation, retention (the demo database prunes frames after three days), access control and transparency towards drivers apply under the GDPR. Refusing a fix has a safety cost if dead reckoning degrades, so the trust score is a research indicator, not a certified integrity figure; TrustNav is not for aviation, maritime or other safety-critical use.

VIII. Conclusion

TrustNav shows that a software layer beside an ordinary receiver can give a fleet operator what neither receiver flags nor fixed probes provide: a per-second, explained trust score for every platform, and a response that keeps a false position from the systems that act on it. On held-out synthetic scenarios the hybrid of explainable rules, an unsupervised model and Galileo authentication detects every attack episode with a median delay of one second, reaches an F_1 of 0.977 with 0.44 % false alarms, while the response layer bounds the operator-visible error to tens of metres instead of kilometres. Replaying real recordings was the decisive test: the rules transferred to a clean drive, but real spoofing defeated the first version until receiver time was checked against the host clock. The next step is hours of real drives on the receiver to be deployed.

Acknowledgment

The authors thank Sofiane Hamrioui, ESAIP Angers, for supervising this project. Map data © OpenStreetMap contributors.

References

[1] Ministère de la Transition écologique (DGITM) and ANFR, "Guide de sensibilisation des acteurs du transport maritime et fluvial et des opérations portuaires aux risques de brouillage du GNSS," Feb. 2024. [Online]. Available: https://anfr.fr/fileadmin/mediatheque/documents/brouillage/ANFR-DGITM_-_Guide_Brouillage_VD_-_2024.pdf

[2] Resilient Navigation and Timing Foundation, "GPS jammer delays flights in France," Sep. 2017. [Online]. Available: https://rntfnd.org/2017/09/15/gps-jammer-delays-flights-in-france/

[3] trans.info, "GPS jammer that brought disruption to Lyon airport," Jul. 2023. [Online]. Available: https://trans.info/en/gps-jammer-that-brought-disruption-to-lyon-airport-353510

[4] Council of the European Union, "Call for common actions in response to Global Satellite Navigation Systems (GNSS) jamming and spoofing threats," 9188/1/25 REV 1, Jun. 2025. [Online]. Available: https://data.consilium.europa.eu/doc/document/ST-9188-2025-REV-1/en/pdf

[5] OpsGroup, "GPS spoofing: final report of the GPS Spoofing Work-Group," Sep. 2024. [Online]. Available: https://ops.group/blog/gps-spoofing-final-report/

[6] Geotab, "Combating GPS jammers," Jul. 2025. [Online]. Available: https://www.geotab.com/blog/combating-gps-jammers/

[7] u-blox, "ZED-F9P FW 1.00 HPG 1.50 release note," UBXDOC-963802114-12826, 2024. [Online]. Available: https://content.u-blox.com/sites/default/files/documents/ZED-F9P_FW100HPG150_RN_UBXDOC-963802114-12826.pdf

[8] Septentrio, "AIM+ anti-spoofing protection," 2025. [Online]. Available: https://www.septentrio.com/en/learn-more/advanced-positioning-technology/aim-anti-spoofing-protection

[9] GPSPatron, "Detecting GNSS threats" and GP-Probe datasheet, 2024. [Online]. Available: https://gpspatron.com/detecting-gnss-threats/

[10] EUSPA, "From testing to operations: Galileo OSNMA service now available to users," Sep. 2025. [Online]. Available: https://www.euspa.europa.eu/newsroom-events/news/testing-operations-galileo-osnma-service-now-available-users

[11] European Commission, "Commission Implementing Regulation (EU) 2023/980 of 16 May 2023 (transitional smart tachograph and Galileo OSNMA)," *Off. J. Eur. Union*, L 134, May 2023. [Online]. Available: https://eur-lex.europa.eu/eli/reg_impl/2023/980/oj

[12] European GNSS Service Centre, "Galileo FAQ: why do we need Galileo OSNMA?," accessed Nov. 2026. [Online]. Available: https://www.gsc-europa.eu/galileo/faq

[13] H. Wang, Y. Zhang, Y. Tan, J. He, S. Zhao, N. Xi, and Y. Shen, "Practical spoofing attacks against Galileo OSNMA with time-synchronization manipulation," arXiv:2501.09246v2 [cs.CR], 2025.

[14] u-blox, "GNSS jamming and spoofing detection at Jammertest 2025," Oct. 2025. [Online]. Available: https://www.u-blox.com/en/blogs/tech/gnss-jamming-spoofing-detection-jammertest-2025-andoya

[15] infiniDome, "GPSdome2," product page and data sheet, accessed Nov. 2026. [Online]. Available: https://infinidome.com/gps-dome-2/

[16] T. E. Humphreys, J. A. Bhatti, D. P. Shepard, and K. D. Wesson, "The Texas Spoofing Test Battery: Toward a standard for evaluating GPS signal authentication techniques," in *Proc. ION GNSS 2012*, Nashville, TN, USA, 2012, pp. 3569–3583.

[17] C. Tang and T. Qin, "GNSS spoofing detection based on LSTM-TNN-CVAE network," in *Proc. APSIPA ASC*, 2025, pp. 1830–1834.

[18] A. Iqbal, M. N. Aman, and B. Sikdar, "A time-series based convolutional VAE for spoof detection in commercial GPS receivers," in *Proc. IEEE Consumer Commun. Netw. Conf. (CCNC)*, Las Vegas, NV, USA, Jan. 2025.

[19] M. Z. H. Bhuiyan *et al.*, "Implementation and performance analysis of a chi-square test based GNSS signal anomaly detection," *GPS Solutions*, vol. 30, art. 154, 2026, doi: 10.1007/s10291-026-02102-z.

[20] J. Shen, J. Y. Won, Z. Chen, and Q. A. Chen, "Drift with Devil: Security of multi-sensor fusion based localization in high-level autonomous driving under GPS spoofing," in *Proc. 29th USENIX Security Symp.*, 2020, pp. 931–948.

[21] A. Kriezis, Y.-H. Chen, D. Akos, S. Lo, and T. Walter, "Real-world spoofing detection and characterization using low-cost receivers," in *Proc. ION ITM 2025*, Long Beach, CA, USA, Jan. 2025, pp. 414–424, doi: 10.33012/2025.19997.

[22] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, Pisa, Italy, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17.

[23] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[24] Google, "GPS measurement tools," 2020. [Online]. Available: https://github.com/google/gps-measurement-tools

[25] Univ. of Vaasa, "VAARA: Vaasa University Jammertest dataset," 2025. [Online]. Available: https://github.com/UniVaasaDigiEco/VAARA

[26] M. Z. H. Bhuiyan, M. Liaquat, and S. Islam, "FGI Jammer Test 2023 Repository," National Land Survey of Finland, 2025, doi: 10.23729/fd-06d27736-45cb-3ca2-aff8-725d42c6caeb.